

Reviewer Pack — Minimal Spanning Tree Diameter and Monomial Ideal Complexity

Entry 4733b7e2ccbe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 07:01:19 UTC

Minimal Spanning Tree Diameter and Monomial Ideal Complexity

Entry ID: 4733b7e2ccbe **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 07:01:19 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Gromov Bound) **Field B** (complexity object): Complexity Theory: Monomial Ideal Complexity

Statement:

[‘For any graph G with n vertices, the diameter $D(G)$ of G is at most polynomially related to the maximum number of generators M for its monomial ideal $I(G)$. Specifically, there exists a constant $c > 0$ such that $D(G) \leq c * \text{poly}(M)$.’, ‘Moreover, if a monomial ideal $I(H)$ has a small diameter, then the corresponding graph H has a low-degree expansion.’, ‘For any graph G with n vertices and diameter $D(G)$, its monomial ideal $I(G)$ can be generated by at most $2^{(c*D(G))}$ generators.’]

Rationale (proposer’s reasoning):

[‘Geometric group theory provides tools for understanding the structure of graphs through their symmetries. The Gromov Bound is a powerful tool that relates the diameter of a graph to its group-theoretic properties.’, ‘If proven, this conjecture would provide a new connection between geometric group theory and complexity theory by linking the diameter of a graph to the number of generators for its monomial ideal.’, ‘This bridge could expose structure in both fields, potentially leading to new algorithms or lower bounds.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 79c51ed381ee1ba1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of seeds, the ratio of the graph’s diameter to the number of generators of its monomial ideal is $\leq c * \text{poly}(M)$, where $c > 0$ is a constant and M is the maximum number of generators. The conjecture is falsified if any

seed produces a ratio greater than $c * \text{poly}(M)$ or if any graph with known diameter $D(H)$ has a monomial ideal $I(H)$ with more than $2^{(c*D(H))}$ generators.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Minimal Spanning Tree Diameter" AND "Monomial Ideal Complexity" - "Gromov Bound" AND monomial ideal generators - "diameter of graphs" IN "monomial ideals"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random graph with n vertices
    n = 10 + random.randint(0, 20)
    G = {i: set() for i in range(n)}
    edges = [(i, j) for i in range(n) for j in range(i+1, n)]
    num_edges = random.randint(int(n * (n - 1) / 4), int(n * (n - 1) / 2))
    for _ in range(num_edges):
        u, v = random.choice(edges)
        G[u].add(v)
        G[v].add(u)

    # Compute the diameter of the graph
    def bfs(start):
        visited = [False] * n
        queue = [(start, 0)]
        max_dist = 0
        while queue:
            node, dist = queue.pop(0)
            if not visited[node]:
                visited[node] = True
                max_dist = max(max_dist, dist)
                for neighbor in G[node]:
```

```

        if not visited[neighbor]:
            queue.append((neighbor, dist + 1))
    return max_dist

diameter = max(bfs(i) for i in range(n))

# Compute the monomial ideal I(G) and find its maximum number of generators
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0:
            return False
    return True

def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def monomial_ideal(G):
    generators = set()
    for node in range(n):
        if is_prime(node + 1):
            generators.add((node + 1,))
    for u in range(n):
        for v in G[u]:
            lcm_val = lcm(u + 1, v + 1)
            if is_prime(lcm_val):
                generators.add((lcm_val,))
    return len(generators), generators

M, _ = monomial_ideal(G)

# Compare the ratio D(G)/M
ratio = diameter / M

# Check if the conjecture holds for this seed
conjecture_holds = ratio <= 2 * n # Example polynomial relationship: c * poly(M) = 2 * n

return {
    "metric_name": "diameter_to_generators_ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"D(G)={diameter}, M={M}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 3 for i in range(5, 8)] # Default list of 30 primes

```

```

results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_ratio = sum(result["metric_value"] for result in results) / len(results)
std_dev = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.2, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.3333333333333333, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.3333333333333333, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.6, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.25, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.25, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.3, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.3333333333333333, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.5, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.2222222222222222, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.2857142857142857, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'diameter_to_generators_ratio', 'metric_value': 0.3333333333333333, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
RESULT: SUPPORTED mean=0.31005291005291 std=0.11793667409626991 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The empirical test has only tested a very small number of instances ($n \leq 15$). This is insufficient to confirm the conjecture, as it may not scale with n and could be an artifact of the particular graphs chosen. The metric does not seem to scale trivially with n , but more extensive testing is needed.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for all tested instances, the ratio of the graph's diameter to the number of generators of its monomial

ideal is within | next: Further testing with a larger variety of graphs and diameters is recommended to confirm the scalability of the findings.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11969
2	preregistration	ollama_remote	glm4:latest	0	0	6440
3	novelty	ollama_remote	glm4:latest	0	0	4771
4	novelty	ollama_remote	glm4:latest	0	0	5407
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14907
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12061
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9444
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10831
9	critic	ollama_remote	glm4:latest	0	0	11725
10	judge	ollama_remote	glm4:latest	0	0	5610

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93163 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/4733b7e2ccbe.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4733b7e2ccbe.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4733b7e2ccbe.tar.gz (if generated)